Università degli Studi di Salerno

Corso di Ingegneria del Software

TrackFit
TPD_TrackFit
Versione 1.0

TrackFit
Sistema di Tracking
Allenamenti

Data: 02/03/2026

Coordinatore del progetto:

Nome	Matricola
Gabriele di Palma	0512119257

Partecipanti:

Nome	Matricola
Yuri Fierro	0512119524

Scritto da:	Gabriele di Palma

Revision History

Data	Versione	Descrizione	Autore
02/03/2026	1.0	Prima stesura	GDP, YF

Indice

1. INTRODUZIONE 4
2. RELATIONSHIP TO OTHER DOCUMENTS 4
3. FEATURES TO BE/ NOT TO BE TESTED 4
   3.1 Funzionalità da testare 4
   3.2 Funzionalità non da testare 5
4. PASS/FAIL CRITERIA 5
5. APPROACH 6
   5.1 Unit Testing con Jest 6
   5.2 Integration Testing con Jest + Mocking 6
   5.3 System Testing con Playwright 6
6. SUSPENSION AND RESUMPTION 7
7. TESTING MATERIALS 7
   7.1 Hardware 7
   7.2 Software 8

1. INTRODUZIONE

1.1. Contesto e motivazioni

L'obiettivo dei test è verificare l'affidabilità e la correttezza delle funzionalità principali del sistema TrackFit, una piattaforma di fitness tracking destinata ad atleti amatoriali e semi-professionisti.

In questo documento saranno verificate le seguenti macro-aree funzionali, selezionate in base alla loro rilevanza critica per l'esperienza utente e la correttezza del sistema:

- Autenticazione e gestione utente (UC1, UC13, UC16)
- Pianificazione e tracciamento delle sessioni di allenamento (UC3, UC4, UC5)
- Gestione abbonamenti e accesso con QR code (UC8, UC9)
- Prenotazione slot coach e corsi in palestra (UC6, UC7)
- Generazione report e sincronizzazione dati (UC11, UC14)

2. RELASHIONSHIP TO OTHER DOCUMENTS

Per la creazione dei test case, ci si riferisce ai seguenti documenti di progetto:

- Requirements Analysis Document (RAD) – TrackFit v1.2: definisce i requisiti funzionali (UC1–UC16), i requisiti non funzionali (NFR) e i vincoli di dominio (R1–R10).
- Object Design Document (ODD) – TrackFit v1.0: specifica le interfacce dei componenti software, i vincoli OCL, i design pattern adottati (Facade, DAO/Repository, Observer, Adapter, State/Strategy) e la struttura a package del sistema.
- System Design Document (SDD) – TrackFit: descrive l'architettura di sistema, i layer applicativi (presentation, application, domain, infrastructure) e le tecnologie utilizzate (Next.js, React Native, Prisma ORM, MySQL).

3. FEATURES TO BE/NOT TO BE TESTED

3.1. Funzionalità da testare

Le seguenti funzionalità verranno sottoposte a test, in quanto rappresentano i requisiti funzionali core del sistema:

- Autenticazione: login, registrazione, recupero password (UC1, UC13)
- Gestione profilo e cancellazione account con conformità GDPR (UC16)
- Pianificazione sessione di allenamento (UC3)
- Tracciamento real-time e terminazione sessione (UC4)
- Recupero sessione dopo crash dell'applicazione (UC4 – crash recovery)
- Sincronizzazione sessione offline (UC5, UC14)

- Acquisto abbonamento con coupon (UC8)
- Validazione accesso tramite QR code ai tornelli (UC9)
- Prenotazione slot coach con pagamento (UC7)
- Prenotazione corso in palestra con gestione lista d'attesa (UC6 – FR25)
- Cancellazione prenotazione entro/oltre le 24 ore (R3)
- Generazione report PDF/CSV (UC11)
- Importazione/esportazione file GPX (FR27)
- Creazione struttura palestra con validazione P.IVA/CUN (UC2 – R8, R9)

3.2. Funzionalità da non testare

Le seguenti funzionalità sono escluse dal presente piano di test per motivazioni specifiche:

- Integrazione con servizi di terze parti (Strava, Google Fit, Stripe): la verifica si limita all'Adapter interno; i servizi esterni saranno mockati.
- Interfaccia di amministrazione avanzata: non ancora completamente specificata nel RAD.
- Notifiche push su dispositivi mobili: dipendono dall'infrastruttura FCM/APNs esterna, verificate con mock dell'Observer.
- Performance e load testing: esulano dall'ambito del corso, sebbene i requisiti NFR-P1/P2 siano stati definiti nel RAD.

4. PASS/FAIL CRITERIA

Le operazioni di test hanno l'obiettivo di individuare eventuali difetti nel sistema TrackFit, così da poterli correggere prima del rilascio.

L'esito di un test è stabilito tramite il confronto con un oracolo, che rappresenta il risultato previsto in base ai requisiti definiti nel RAD e ai vincoli OCL specificati nell'ODD.

- Un test si considera superato (PASS) se l'output effettivo coincide con quello atteso dall'oracolo, incluso il rispetto delle postcondizioni OCL.
- Un test si considera fallito (FAIL) se l'output differisce da quello previsto, oppure se una precondizione OCL non viene rispettata, o se il sistema lancia un'eccezione non gestita.

Il testing sarà ritenuto adeguato quando:

- Tutti i requisiti funzionali legati alle funzionalità selezionate nella sezione 3.1 sono coperti da almeno un test case.
- La copertura del codice (branch coverage) per i moduli critici (UserManagement, WorkoutManagement, SubscriptionManagement) raggiunge almeno l'80%.
- Tutti i test case di livello System (end-to-end) relativi ai flussi UC1, UC3, UC4, UC8, UC9 risultano PASS.

5. APPROACH

Per garantire la qualità e l'affidabilità del sistema TrackFit, verrà adottato un approccio strutturato al testing, articolato in tre livelli principali: unit testing, integration testing e system testing. Ogni livello utilizza strumenti specifici adatti allo stack tecnologico del progetto (TypeScript, Next.js, React Native, Prisma ORM).

5.1. Unit Testing con Jest

Il primo livello si concentra sulle unità fondamentali del codice: funzioni di validazione, metodi dei service layer, logica OCL (precondizioni/postcondizioni) e utility (es. calcolo penale cancellazione, verifica unicità P.IVA/CUN).

Per questa fase verrà utilizzato Jest, il framework di testing standard per progetti TypeScript/Node.js. Jest permette di:

- Verificare la correttezza delle singole funzioni in isolamento (es. validazione email, calcolo penale 50% su cancellazione tardiva R3).
- Testare i vincoli OCL implementati come guard nei service (es. avvioTracciamento richiede stato sessione = Pianificata).
- Integrare facilmente i test nei workflow CI/CD, con rilevazione precoce degli errori.
- Misurare la code coverage tramite il reporter integrato di Jest (--coverage).

5.2. Integration Testing con Jest + Mocking

Il secondo livello verifica l'interazione tra i componenti del sistema: API Routes di Next.js, service layer, repository (Prisma ORM) e moduli di notifica.

Verrà utilizzata la libreria jest-mock-extended per mockare il PrismaClient, simulando il comportamento del database MySQL senza richiedere una connessione reale. Questo approccio consente di:

- Verificare la logica di interazione tra service e repository (es. acquistaAbbonamento verifica unicità abbonamento attivo, salva Coupon, aggiorna stato Utente).
- Testare scenari di errore (es. database non raggiungibile, coupon scaduto, QR già utilizzato).
- Simulare il comportamento dell'Observer per i canali di notifica (email, push) tramite mock delle interfacce INotificationChannel.
- Verificare gli Adapter per integrazioni esterne (Stripe, Strava) senza effettuare chiamate reali.

5.3. System Testing con Playwright

Il livello finale si concentra sulla validazione del sistema nella sua interezza, simulando scenari d'uso reali tramite automazione del browser. Per questa fase verrà utilizzato Playwright, uno strumento moderno per l'automazione end-to-end che supporta Chromium, Firefox e WebKit.

Attraverso Playwright sarà possibile:

- Simulare il comportamento dell'utente sull'interfaccia web (Next.js): navigazione, compilazione form, click su bottoni, verifica risposte dinamiche.

- Testare i flussi completi più critici: registrazione → login → pianificazione sessione → tracciamento → completamento (UC1, UC3, UC4).
- Verificare il flusso di acquisto abbonamento con coupon e la successiva generazione del QR code per l'accesso ai tornelli (UC8, UC9).
- Testare il comportamento del sistema in caso di sessione offline e successiva sincronizzazione (UC5, UC14), simulando la perdita di connessione tramite le API di Playwright.
- Validare la generazione e il download dei report PDF/CSV (UC11).

6. SUSPENSION AND RESUMPTION

I test potranno essere sospesi nei seguenti casi:

- Blocco critico dell'ambiente di sviluppo (database non raggiungibile, build fallita): i test verranno sospesi e ripresi una volta ripristinato l'ambiente.
- Difetto bloccante identificato che impedisce l'esecuzione di test dipendenti (es. bug nel modulo di autenticazione che blocca tutti i test a valle): i test dipendenti vengono sospesi e ripresi dopo la correzione del difetto.

In assenza dei suddetti eventi, non sono previste sospensioni ordinarie. Il testing riprende automaticamente dopo la risoluzione della causa di sospensione, senza necessità di ricominciare dall'inizio grazie al supporto della pipeline CI/CD.

7. TESTING MATERIALS

7.1. Hardware

Per l'esecuzione dei test saranno utilizzati i seguenti dispositivi:

- PC di sviluppo (Windows/macOS/Linux): per l'esecuzione di unit test, integration test e system test tramite browser headless Playwright.
- Smartphone Android e/o iOS (opzionale): per la verifica manuale dell'interfaccia mobile (React Native/PWA) sui casi d'uso principali.

7.2. Software

Lo stack software richiesto per l'esecuzione dei test è il seguente:

Componente	Strumento / Versione
Runtime	Node.js >= 18.x
Framework testing unit/integration	Jest >= 29.x
Mock Prisma ORM	jest-mock-extended >= 3.x
Framework testing end-to-end	Playwright >= 1.40
Database (test environment)	MySQL 8.x (Docker container)
ORM	Prisma >= 5.x
Frontend	Next.js >= 14.x (TypeScript)
CI/CD	GitHub Actions
Coverage report	Jest --coverage (Istanbul)

L'ambiente di test viene isolato dall'ambiente di produzione tramite variabili d'ambiente separate (.env.test) e un database MySQL dedicato, istanziato come container Docker per garantire riproducibilità e indipendenza dei test.